Implement the Actor model from scratch

1. Let's start with the purpose: where and why does it make sense to use the Actor model?

   > - A server application divided into components that communicate with each other, where we want to avoid concurrency errors.
   > - A system that we want to scale: start small, grow if needed, either automatically or by adding more servers.
   > - Some unconventional uses, which I nevertheless see often in student projects and also in my own past work, involve using actors to model neurons, non-playable characters in video games, or crowd simulations.

2. How do actors help?

   > - An actor sits on top of a component as the only gateway to it.
   > - Each actor works on a separate thread (*).
   > - Processing within the actor is sequential.
   > - Actors communicate only by sending messages to each other.
   > - Actors can spawn more actors.
   > - You may find the code needed for using actors to be overly verbose, but there is another side to it: the way we use actors feels intuitive. It is like having a conversation. An actor sends a message to another actor and then awaits the response (or not; maybe that message does not require a response, and the actor just switches to talking to someone else). Since we humans talk to other people all the time, giving tasks, asking questions, responding to questions, reacting to new information, and so on, programming logic for actors might be easier for us than using another approach, such as an effect system in functional programming.

3. In this lecture, we will build an actor system implementation from scratch:

   > - ... but because of the time limit, we will do it only in theory, on slides.
   > - Code examples are in Scala 3.

4. The minimal functionality of an actor:

   > - **The "fire and forget", `tell`, or `!` method of sending messages**: There is no guarantee that a message actually reaches the receiver and is processed successfully; some messages can be lost. If a message is very important, the sender should require a confirmation in the form of a response

message and, if that message does not come, may send the original message again.

- **Untyped**: A message can be anything, a primitive type or an instance of any class.
- **An internal FIFO queue for messages**: messages are guaranteed to be processed in the order in which they arrive.
- **A message is NOT processed immediately**: it lands in the queue and waits to be processed.
- **A heartbeat**: Therefore, we need a job, on a separate thread or a virtual thread, that runs the processing at regular intervals. I would like to call that job the `heartbeat`. At regular intervals, it checks whether there are messages in the queue and, if there are, takes them out and processes them.
- In Scala, the thread pool on which tasks run is called `ExecutionContext`. A `heartbeat` can be implemented as a task that resubmits itself to the `ExecutionContext` after every run.
- Ideally, that **execution context is serial**, and we do not use any other form of concurrency in the processing.
- **The `onMessage` function** is called during processing for every message in the queue. A message is removed from the queue before it is processed. Processing ends when there are no more messages in the queue.
- One catch: a message can arrive during processing. The implementation may decide whether such a message should be processed in the same round of processing or wait for the next one.
- **Indirect responses**: `onMessage` does not return any result (`def onMessage(msg: Any): Unit`); instead, results are passed with messages.
- `Any => Unit` is heavily criticized in FP circles in Scala.
- Also, **an actor needs references to the components** that it is supposed to control. Ideally, these components should not be accessed by any other actor or by any external component.
- How do we **implement the `onMessage` function**? One way is **by subclassing**. We start with an interface, or in Scala a trait, and then a class that is still abstract, because it does not provide the `onMessage` implementation, but which extends that trait and implements the `!` method, the queue, and all the logic needed for processing messages, except for `onMessage`. Let us call the trait `Actor` and the abstract class `BaseActor`.
- The reason for the existence of **the `Actor` trait** is that we can now use it as a reference and give it to other actors. For now, that trait **exposes only the `!` method**, enabling another actor to send messages to this one. There will be more methods coming as we develop the system, but it will always remain a rather short trait. That is also one of the qualities of the Actor model: since communication goes via messages, we do not need to expose many methods.
- Now the user can subclass `BaseActor`, creating a class that implements `onMessage` however they want. That user class can also be the way we give the actor references to its underlying components.

5. Basic actor workflow

    - The actor is **created with a reference to the underlying component** (for example, a database).
    - It is **initialized: from now on, the `heartbeat` runs**.
    - The actor **receives a message through `tell` / `!`**. The message has a type (= information about how to process it), additional data, and a reference to the sender actor.

- The message is put in the queue.
- The heartbeat triggers the processing.
- The message is taken out of the queue and given to `onMessage`.
- `onMessage` checks the type and, if it knows the type, performs an operation according to the data in it (for example, queries the database).
- If the query takes time, the processing simply awaits.
- When the result comes back, it is sent to the sender actor, wrapped in a response message.

6. Communication between actors

- The simplest way: **an actor sends its reference** in a message to another actor.
- Okay, but **how does the first actor know** where to send its message?
- Ideally, an app using the Actor model does not have any top-level components that are not actors. At the beginning, we **create all the necessary components and actors, give the right actors references to the components and also to other actors**. And from now on, all the work in the app is done by the actors.
- But if we were **to distribute actor references through constructors**, that **would cause the circular initialization problem**.
- Instead, we can first **create the actors and then send them references to other actors in special messages** that do not require a response. Let us define such a message: `enum SystemMsg { case Register(actorRef: Actor) }`.
- **System messages** are a special kind of messages that actors send to each other to maintain the system. There are other such messages. For example, when I talked about the workflow of an actor, I did not mention how an actor is stopped; that is also done with a system message, and this is the place to tell you about it. Typically, we have a system message `Close`, and then another one, called `Shutdown` or sometimes `Kill`. When an actor receives `Close`, it means it can stop gracefully: it has some time for it, so it can, for example, process its remaining messages. `Shutdown` means "close right now"; it is like `kill -9` for a Unix process.
- System messages can be sent to actors by the user, but much more often they are exchanged between actors in response to other system messages. They are like a separate conversation taking place in between the regular work. For now, remember that we can use them to register actors and to close actors. There will be more.
- Take note that at the moment our actors can take a message of any type, so a `SystemMsg` can go through the usual channel: `!` -> queue -> `onMessage`, just like a regular message. When we had only regular messages, the implementation of `onMessage` was entirely up to the user. Now it is not: we want to handle system messages in our own code and leave only the handling of regular messages to the user. This can be done by implementing `onMessage` in `BaseActor` itself. That implementation will handle system messages, and the user's implementation, in the subclass, will call it for any message it does not handle directly. That is: the user's code starts to process a message, checks its type, and decides it does not recognize that type; instead of dismissing it immediately, it passes it to `super.onMessage`, because it might be a system message that will be recognized there.

7. Internal state

- Let us assume we receive `Register(actorRef: Actor)`. We do not want to just process it, send a

message to that actor immediately, and then forget we ever had that reference. No, we want to store that reference, so that we can use it in the future.

- Let us add a `private var refs: Map[String, Actor]` to our `Actor`. This is a map of some identifiers to actor references. We cannot do without identifiers, because if we get more than one actor reference, we need to be able to distinguish them. After all, we will send different messages to an actor taking care of a database than to an actor taking care of an internet connection.
- We can assign identifiers by type. In `onMessage`, we can check the exact subclass of the `Actor` instance we get and store it in the map as "database-actor" or "internet-actor". But that is clunky. There may be many database and internet actors, and we still need to distinguish between them.
- Instead, we let them identify themselves. We add `val id: String` to `Actor`. It is set in the constructor and never changes after creation, so it can be public.
- In fact, we can generalize this. There might be other, regular messages that want the actor to modify its internal state, so that the next time a specific message comes, the actor can access that state and make a specific response.
- In our current implementation, we can simply leave it to the user. A subclass of `BaseActor` might contain any additional private mutable data.
- But there is one more thing: internal state affects how `onMessage` works. Now, with every processed message, `onMessage` might need to check if there is something in the internal state that makes it take another logic branch to create the result. Sometimes that is okay: we process a message, we determine its type and data, and then we take the internal state into account. But sometimes it actually makes sense to switch to another implementation of `onMessage`.

8. Behavior switch

- Imagine that there is an integer in the internal state. That integer can change between, say, 0, 1, and 5, depending on the incoming message. And every time a message is processed, the value of that integer affects the outcome.
- Instead of having that integer, we can simply have several versions of `onMessage`. A message comes, it is processed, and as a result the actor decides to switch to another `onMessage`. The signature becomes `def onMessage(msg: Any): (Any => Unit)`, that is, the result of processing is the function that will be used for processing the next message. It can be the same function; usually it is, but it does not have to be.
- In practice, the user will now write additional functions with this signature, and in each of them, the returned value will be either the current function or another one. They will all take the place of `onMessage`.
- In other words, instead of the one implemented `onMessage`, we now have many possible behaviors. The actor becomes a state machine. A message can affect its behavior, moving it to another place on an abstract behavior graph.

9. Typed actors

- Let us take a step back. Until now, we maintained that anything can be a message. An actor system that accepts anything as a message is called "untyped". We basically threw the type system out of the window; it could not help us. And in Scala, especially in Scala, the type system can help us a lot, if we let it.
- Let us change that now. We add a type parameter to the `Actor` trait and to its implementation, `BaseActor`: `trait Actor[Msg]`. Now, when we create an actor instance, we have to specify what type

of messages it accepts. The `!` method will accept only instances of that type.

- In Scala 3, we finally have enums (we did not have them in Scala 2), and they are a very straightforward feature for defining messages for typed actors. An enum has cases; those cases can be singletons, but they may also have their own data, meaning that two instances of one enum case may still differ from one another; the enum itself may also have data fields. Think of it as if the enum were a superclass and each case were a subclass of that superclass. For example, we can create:

```
enum Msg {
  case ReadFile(sender: Actor[Msg], filePath: Path)
  case FileContents(contents: String)
}
```

The first message will be sent to an actor that takes care of the file system. That actor will process it and, in its `onMessage`, read the contents at `filePath`, and then it will send those contents to the `sender`, wrapped in the second message.

- Sometimes it might make sense to have a deeper hierarchy of messages. Enum cases cannot have their own sub-cases, so we cannot use enums for that, but we can go back to the feature used in Scala 2: sealed traits, case objects, and case classes. Traits in Scala can have data fields, and of course they can have sub-traits and sub-classes, and unless those sub-classes are final, they can have their own sub-classes. A non-final case class is a bit frowned upon in Scala circles, because a case class is supposed to be just a simple data structure, but it is totally okay to have a tree of sub-traits or sub-classes that are not case classes, with case classes and case objects only as the leaves of that tree. An instance of a subclass is accepted by an actor with the superclass parameter, and we also gain the ability to create actors that accept only a subset of all those messages; thus we take advantage of the type system.

11. Typed messages
    - We also need to decorate the queue with the `Msg` type... and that creates a problem. The queue no longer accepts system messages, since they are a type of their own. There are two ways to overcome this problem.
    - The first is to use union types in Scala 3, i.e. `Queue[Msg | SystemMsg]`. And then, instead of directly calling `onMessage` for every message, we first check if the message is a `Msg` or a `SystemMsg`: for a `Msg` we call `onMessage`, and for a `SystemMsg` we call a separate `onSystemMsg`. So, you can see, we have done away with the idea of having a superclass and a subclass `onMessage`; now we simply process system messages separately.
    - But there are problems with this approach. First, union types are not very popular in programming languages. If we wanted to do the same in Java or Kotlin, we could not. Sometimes, instead of union types, we can use an interface, and we could do it here:

```
trait AbstractMsg
enum Msg extends AbstractMsg { ... }
enum SystemMsg extends AbstractMsg { ... }
val queue = Queue.empty[AbstractMsg]
```

That is not too bad, but it requires the user to add this `extends AbstractMsg` to every message type.

- Another possibility is to have two queues, one for regular messages and one for system messages. Then, the system message queue sends its messages to that other `onSystemMsg` function; processing

system messages becomes completely separate from processing regular messages, and we can hide it better from the user.

- On the other hand, it breaks our promise that messages will be handled in the order in which they arrive. We have two queues now. We could add timestamps to the messages, so `Queue[(Msg, Long)]` and `Queue[(SystemMsg, Long)]`, and this way we would know in what order to process messages. We can also simply say that the promise concerns only regular messages. System messages, which are mostly hidden from the user and anyway seem to be of higher priority, may all be processed first, with regular messages only after that.
- But there is one more problem. Look what happens to the `Register` message: `case Register(actorRef: Actor[Msg])`. Yes, we need to parametrize the system message now, otherwise the code will not compile. We could parametrize the case, as in `case RegisterMsg`, but it is obvious that we will need this parameter for other system messages too, so we might as well parametrize the whole enum:

```
enum SystemMsg[Msg] {
case Register(actorRef: Actor[Msg])
}
```

12. Exhaustive pattern matching

- Okay, but why go through all this trouble? Surely not because there are a bunch of functional programming enthusiasts on the net who criticize us and we want them to stop. There is a better reason. It is called exhaustive pattern matching.
- Scala has a very powerful pattern matching feature. I would even say that it is not only one of the major features in Scala, but the main feature. It is everywhere: an alternative to a simple if/else, destructuring and verifying complex data structures, defining partial functions.
- With untyped actors, anything can be a message, so there is always a possibility that a message our `onMessage` does not know how to handle is still a valid message. It will get ignored, and that might actually lead to nasty bugs.
- But with typed actors, the compiler can perform the exhaustive pattern matching check. That is, the compiler knows all the possible message cases in an enum, and all the possible subclasses in a sealed trait hierarchy, so even if my logic in `onMessage` is pretty complicated, the compiler can tell me whether that logic covers all possible cases or whether I missed something. And if I missed something, the compiler can even give me a hint about what it is.
- It also works the other way: in IntelliJ IDEA, I can easily generate match cases for all enum cases, then fill out what happens if the message is of that case, and I am done; I am now 100% sure that the actor will react in the right way to every possible message.
- It becomes even more important nowadays, as more and more code is written by AI. The type system, together with compiler errors and warnings, becomes a very useful tool for AI coding agents. If the AI agent makes a mistake and sends a wrong message to an actor, the code will not compile, and the error message from the compiler will say clearly why: because the actor accepts only such messages, and this message is different. With untyped actors, the code would compile without a warning, and we would learn that something is wrong only at runtime or in unit tests.

13. Sub-actors

- So far we have gone through 4 of the 5 points I mentioned at the beginning: 1. An actor sits on top of a component as the only gateway to it; 2. Each actor works on a separate "thread"; 3.

Processing within the actor is sequential; 4. Actors communicate only by sending messages to each other.

- The last point is "Actors can spawn more actors". It might seem simple, but talking about it will probably take us the rest of this lecture.
- It ties directly to point 3, "Processing within the actor is sequential". Ideally, we should not use any other form of concurrency in our `onMessage` logic: no `Future/Promise`, no new threads, because it breaks the very reason why we use actors, namely that they save us from concurrency errors. If we break that rule, we might as well not use actors at all.
- But that is a serious limitation, isn't it? What if, in response to a message, the actor needs to send a complex query to the database and wait for the answer? Or what if there are hundreds or thousands of messages coming in, most of them unrelated to each other, and yet we process them sequentially instead of in parallel?
- The answer to that is more actors.
- Instead of performing the task by itself, the actor can monitor the situation: the number of messages in the queue, the average time spent on processing a message, the percentage of particularly processing-heavy messages, and so on. Take note that an actor does not need to be only reactive. The idea is that it will process the incoming messages, yes, but there is nothing in the concept preventing us from implementing some other logic, aside from processing messages. We will get back to it later (statistics and logging).
- When the actor notices there is too much work, it may delegate. That "noticing" can be the result of one of the statistics getting too high, or it can simply be a result of processing a system message from another entity. Let us call that system message `Spawn(config: ActorConfig)`, where `ActorConfig` is a simple case class that describes the concrete subclass of the sub-actor and its initial state, passed to the constructor.
- The actor may also have that data from the beginning. We give it to it at construction time, and implement the logic saying "hey, if the statistics show X, then you can spawn a sub-actor of subclass Y".
- When the sub-actor is spawned, the `onMessage` processing changes. That change might either be hardcoded into our `onMessage`, i.e. "if the message is X, but we have a sub-actor Y, then...", or (as we discussed earlier) we may switch from the default `onMessage` to another one, to be used when there are sub-actors.
- That new logic, instead of processing message X, sends it to the sub-actor, and either awaits the response and then sends it to the original sender, or gives the sender's reference to the sub-actor, so that even this part of the processing is delegated.
- If needed, an actor can spawn many sub-actors to process messages that do not get in each other's way. This is the right way to have concurrency in the Actor model implementation.
- The same also works in the other direction: when the spike of activity is gone, the parent actor may decide to close some or all of the sub-actors (we have already talked about closing actors; that mechanism also applies here).
- Or, if we anticipate another spike, the parent actor may pause sub-actors instead of closing them. Closing and creating actors is expensive. Instead, the actor may send a system message `Pause`, and then the sub-actor will simply stop calling `onMessage` for new incoming messages, until it receives the `Unpause` system message.

14. Logging and statistics

- Let us take a small step back. I said that **the decision to spawn a sub-actor might be automatic** and based on gathered data about the performance of the system.
- Therefore, we need a way to gather that data.
- Here we are moving into the user's territory: **gathering statistics and other data about how the system performs can be very specific to how the user builds and uses the system**, but implementing this functionality is also such a common need that you might want to help your users and meet them halfway.
- Let us create a **special actor, `LogActor`**. It will handle system messages such as `AverageProcessingTime(ms: Long)`, `MessagesPerSecond(number: Long)`, `LongestProcessingIn(ms: Long, between: (Long, Long))`, and others like that. Please note that, apart from the actual numbers, every such message should carry a timestamp and a sender reference, because it should be logged to a file in case it can be important for debugging. We can also use all of that to build a statistical model at runtime, as the `LogActor` internal state: for example, we can track the average processing time of an actor and, if that time increases over some boundary, decide to spawn a sub-actor for that actor.
- And yes, apart from messages carrying statistics, we can also **use `LogActor` for errors and warnings**: those will be logged to a file, and the actor system may also react to them and try to fix the situation.
- Oh, and about how it works on the sender's side. Every heartbeat, we call `onMessage`, but that does not have to be the only thing we do. We can make a timestamp (`System.currentTimeMillis()`) before the processing starts, and then, after it is done, make timestamps before and after each type of message is processed; we can keep those intervals in the internal state to calculate an average over a given time period. Also, we can wrap `onMessage` in a try/catch, and if the processing fails with an exception, catch it and send an error message instead. And also, if an actor receives a message it cannot handle (which, mind you, it should not receive in the first place), we can turn that into an error message too.

15. Restart-on-error and serialization

    - What should happen when an actor produces an error message? Especially if this is not a one-time issue but happens regularly?
    - As we already discussed, it is possible to close an actor. But what if we actually need it?
    - The solution to this is as old as IT itself: we can close the actor and start it again. Hopefully, this will fix the problem. If not, well, at least we tried. The good thing is that restarting is pretty straightforward and can be done automatically, while more complicated solutions usually require the whole system to stop and someone to look at the logs. Someone, meaning a human or, nowadays, an AI agent.
    - But we need one more feature before we can do it: we need to be able to save the actor's state and then load it back. This means we need to implement the following:
        1. First, we need to serialize the internal state of the actor: the mutable state, but also its id, its type, and everything else that is needed to restore it later. The serialized format can be whatever we choose, but probably JSON. We can implement `toJSON` in the actor as a private method, using one of a dozen Scala JSON libraries, but please note that every piece of data held in the internal state must be serializable as well. That is actually quite a big limitation, because we cannot serialize and deserialize direct JVM references. For actors, we can save ids instead (and we will come back to that soon), but for other components,

the user must come up with some solution that will let us store some information now and reconnect the actor to the component later.

2. We may also want to serialize the unprocessed messages, and then deserialize them on the other end and add them back to the queue. That is one option. Another option is to process them all before serialization, which means that the system message causing the serialization must be processed out of order, as the last one. And yet another option is not to worry about it; after all, one of the fundamental ideas in the Actor model is that not every message must be processed, and some might be lost for different reasons.
3. Second, we need to write down that serialized data. It can be done by the actor itself: we can, for example, give each actor a path to a directory reserved for that, and the actor's id plus `.json` will be the name of the file in that directory. But there is another option: we may send that JSON data in a message to a special actor that takes care of writing it to a file. The first option is more direct, but it requires every actor to have access to certain data and to contain certain logic that the other option does not require.
4. Third, we need something like an `ActorFactory`, or an `ActorBuilder`. Given the JSON string, that builder should be able to create a new actor of the same type, with the same id and the same internal state, and maybe even with the same unprocessed messages in the queue. Plus, in the case of some actor types (a type here being a subclass of `BaseActor`), the actor requires some special initialization after creation, through a regular message, `Init(...)`. That message should carry information that could not be given to the actor directly in the constructor, such as references of the actors or other components that the new actor should connect to.

- A terminological remark: this "restart-on-error" design is also known as "let it crash". It's been developed in Erlang and then, in Akka, it's a base for so-called "supervisor hierarchies" where a parent is responsible for restarting its children, but that child might also have its own children which in such case need to be restarted as well, even if they work well.

16. Parental privileges (close / restart)

- Finally, we need to answer the question who has the right to restart another actor. It feels that it shouldn't be any actor, right? It would be too easy to make a mistake and close an important actor simply by sending a wrong message to it from any source. "I wanted to ask about the size of a database table but instead I crashed everything, ooops".
- The simplest answer, although a bit limiting one, is that only the parent of the actor can close it. So, we build a hierarchy of actors. We start with just one, created when the system starts, and all the other actors in the system are its sub-actors, and those sub-actors can have their own sub-actors. The sub-actor, if anything wrong happens, may send a message to its parent about it, and the parent may decide to close it. Also, `LogActor` can send a message to the parent to restart its child.
- On top of that, it is always possible that an actor will just go silent because of a problem - it will not send out messages or react in any way to the ones it receives. We can remedy that by making the parent send a `HealthCheck` message to the sub-actor every, let's say, 100 heartbeats, and expecting `HealthCheckResponse`. If the actor fails to respond for, let's say 3 times, it means something is wrong with it and the parent may decide to restart it.

17. `ActorSystem` and discoverability

- Let us go back to the issue of how actors get references to other actors. When we talked about a system with only a few actors, it was easy to just create them manually and send their JVM references to one another. But when actors can spawn more actors, this is no longer possible.

- Of course, we may delegate that logic to the parent actor. It spawns a child and then sends it a message with references to other actors. That means that the parent actor would have to have those references before the child is spawned. That is not very scalable. We are basically forced to send references to actors just in case they are needed by sub-actors that might never have been created.
- An alternative solution is to have something like a storage of actor references. Let us call it `ActorSystem`.
- Just as every actor has an optional reference to its parent, in case it is spawned by a parent actor, now it will also have an optional reference to the actor system. We will give it to it in the constructor.
- And every time an actor spawns a sub-actor, it will also give it its reference to the actor system.
- Here is the important part: the moment the actor is initialized, it sends the actor system a message with its own reference, `Register`. The actor system stores it and then allows for its retrieval.
- When the actor closes, it sends the `ActorSystem` an `Unregister` message. Or, the parent actor can do that, because the reason for closing might be that the sub-actor is unresponsive.
- But how? Well, first of all, since the actor system communicates with all other actors through messages, and we do not want to duplicate our code, it can also be an actor. In fact, it can be the top actor, the one at the top of the hierarchy, the parent of all other actors. This way, we simply implement the logic of carrying the actor system reference through spawning. Instead of creating new actors directly, we send the `ActorSystem` a message "spawn an actor of the class X with the following data". After doing that, the actor system will give the new actor its own reference, both as the parent and as the actor system. And when that actor spawns its own sub-actor, it will give it a reference to the actor system.
- Now, the retrieval part. Let us say we have a business logic actor, and a database actor, and the actor system on top. If they want to get references to each other, they need to ask the actor system for them. (Please note that this does not mean that the old way of directly giving out actor references in constructors is bad; it is still perfect for small systems.) So, `BusinessActor` must send the `ActorSystem` a system message `AskForRef(...)`, and inside it, there must be some information that helps the `ActorSystem` decide that it needs to respond with `Ref(refToDatabaseActor)`. As in, not a reference to any other actor: only a reference to the database actor is the right answer.
- One way to do it is to give actors correct ids: for example, `business-1` and `database-1`. Then, the `BusinessActor` might send `AskForRef` with something like `database-*` inside. In `ActorSystem.onMessage`, for `AskForRef`, we will parse it and learn that we need to send back a ref to an actor with an id starting with `database-`.
- Another way, less used as far as I know, but still possible, is to register an actor in the `ActorSystem` not only with its id and reference, but also with a list of tags. So: `RegisterActor(actor: Actor, id: String, tags: List[String])`. And now, `AskForRef(...)` may come with a tag, or a list of tags, and in `ActorSystem.onMessage` we will look for a registered actor that has all those tags on its list.
- If there is no match for an actor, we may simply not answer (that is always an option), or we may send a response system message like `NoRef(reason)`. That message may also go to the `LogActor`.

Messages might be simple enums or complicated hierarchies of traits and case classes

18. - The implementation so far

- Let us recount what we have talked about so far:
- We learned how to build **an actor system that is composed of actors of different subclasses**, with different logic for handling messages and different internal states.
- **They can communicate with each other** by sending regular messages and their own internal system messages.
- Regular **messages might belong to simple enums, but they can also be complicated hierarchies of traits and case classes** implementing those traits. They do not have to be sealed, but if they are, we can profit from exhaustive pattern matching. Besides, the fact that the actors accept only messages of a given type already helps, because the compiler can catch many bugs that would otherwise only surface at runtime.
- **The actors can spawn their sub-actors**, and that can help if they have too much work, or if they need some special task to be performed. This is basically how actors handle concurrency, because inside one actor we want the logic to be executed sequentially.
- **To keep track** of whether we need to spawn more actors, **we can have a special `LogActor`** that other actors send work statistics to, and **it can respond with a request that they should spawn a sub-actor** to help them in their work. The same actor can also store and write down information about errors and warnings.
- Also, if it seems that the system has grown too much and there is not enough work for it, the same `LogActor` may advise a parent actor **to pause or even close its sub-actors**. An actor may also be restarted, that is, closed and created again, if it malfunctions.
- For that, we need **the functionality of serializing** the internal state (and optionally also the messages in the queue), and then deserializing them back.
- Finally, **there is the top actor, `ActorSystem`**, which stores references to all the actors in the system and helps connect any two of them.

19. Communication across actor systems

- All this is already quite a lot, but we still have not touched one important part. At the beginning I mentioned that we want an actor system to be scalable: start small, grow if needed, either automatically or by adding more servers.
- Indeed, we can simply start another application, on the same server or on another server, and let them talk to each other through an API.
- But how can an actor send a message to an actor on another server? So far, we used direct JVM references, and those only work if both actors are inside the same JVM application.
- Ideally, we would like `actorRef ! SomeMsg(...)` to work exactly the same whether the actor is local or remote. It is called "location transparency". For that, we need to take `actorRef` to a higher level of abstraction, so to speak. It cannot be a direct JVM reference.
- Let us introduce a trait, `ActorRef`, with an abstract method `!`. Then, let us make two concrete subclasses: `LocalActorRef(actor: Actor)` and `RemoteActorRef(mySystem: ActorSystem, refSystemId: String, actorId: String)`. In practice, it can be even more complicated than that, but I just want to explain to you how it can work.
- Both those subclasses override `!` with different implementations. `LocalActorRef` simply calls `!` on `Actor`. `RemoteActorRef` sends a system message to its actor system: `Send(msg: Msg, remoteSystemId: String, remoteActorId: String)`. As you can see, that system message

wraps the original regular message, plus it carries the systemId, that is, the id of the other actor system, in which the receiver actor lives, and the receiver actor id.

- The actor system on the sender's side receives the system message. Now it needs to send it further, to the other system.
- In order to do that, it needs to already know about that other actor system. This is done by the external API: when an actor system application starts, it should read its configuration file, or learn in some other way, that there are other actor systems online. The new actor system sends the system message `Register` to those actor systems through the API (by the way, that also means system messages need to be serialized and deserialized, just as we discussed before for the internal state and regular messages). In return, it gets a response system message with the systemIds of those other actor systems. This is basically like when we spawn and register individual actors; we can reuse the same logic to some extent.
- Now, when an actor wants a reference to another actor and asks the actor system about it, the actor system might respond not only with an `ActorRef` to an actor in its own system, but it may also ask around. Because maybe we do not have the right actor here, and for some reason we do not want to spawn it, but there is already one on another server. Communicating with it will be a bit slower, but maybe it is worth it.
- In fact, optionally, the actor systems may communicate with each other before any such request comes, sending to each other information about what actors they have. So when a request comes, the actor system does not have to ask through the API; it can answer immediately with the right `ActorRef`.
- It will be a `RemoteActorRef`. When the actor that requested the communication decides to send a message to that other actor on another system, the message will first be wrapped in a system message `Send`, and it will go to the local `ActorSystem`; there it will be serialized, sent through the API to the remote actor system; and there, the other actor system will check the `remoteActorId` and finally send it to the right actor. Of course, many things can go wrong in the middle of the process, and at every step we have to be ready to handle errors. But, again, this is similar to communication between actors in the same system, and we can reuse the same logic.

20. Optional: Sub-actors on other actor systems

- So, we can now have many actor systems that communicate with each other. On top of every actor system sits a special actor that we also call `ActorSystem`, and this actor is at the same time the parent and ancestor of all the other actors in the system. The diagram here looks like a row of Christmas trees.
- But, just to finish this part quickly: it can be more complicated than that. Since so much logic is the same for local and remote communication, there is basically nothing stopping us from sending a `Spawn` message across systems. It means that an actor on one system can get an `ActorRef` to an actor on another system, and ask it to spawn a sub-actor and give back a reference to that new sub-actor to the actor on the other system. All it requires is adding the ability to send system messages remotely, not only between actor systems, but also between regular actors. That is not really complicated.
- The complicated part is that, if that actor is the only one interested in the sub-actor on the other system, we might actually want to tighten the connection between them and make them parent and child. That will require one more system message, `RemoteSpawn`. The first actor sends that message to the remote actor system (remember, actor systems are actors, so they can communicate with regular actors as well, not only with actor systems). Then, the remote actor

> system spawns a new sub-actor, but instead of making itself the parent, it gives the new sub-actor's `ActorRef` to the original actor, as its child, and the original actor's `ActorRef` to the sub-actor, as its parent. The new sub-actor will still have a local `system` reference, but it will communicate with its parent through an `ActorRef` that will, underneath, be a `RemoteActorRef`. To make this possible, we need to switch from direct JVM references for parents and children to our higher-level `ActorRef`s. But once that is done, that is basically it.

21. Optional: the `ask` method

> - Until now, we have been talking about implementing the core Actor model. Communication in such a model is done through "fire and forget" messages. We have no guarantee that the message will reach the target, and no guarantee that we will get a response. What is more, another rule is that the actor's logic should be sequential, and concurrency should be handled by spawning new actors. Specifically, there should be no `Future` logic inside `onMessage`. All that helps us keep the logic moderately simple and avoid a subset of concurrency errors. But there is a drawback.
> - I mentioned before a recurring process: an actor sends a request and awaits either a response, or a confirmation that the request was successfully handled, or an error message. With `!`, this must be asynchronous. In the `onMessage` logic, we need to have a case for each of those possible responses, and such a response message must carry information that it is a response from a particular actor to a request sent before. Or at least the other actor's id, and then we rely on the first actor remembering, in its internal state, that it has sent a request to that actor.
> - It is messy, and it happens often enough that Akka and Pekko came up with a solution; it breaks the rules I talked about just a moment ago, but only slightly.
> - The new method is called "ask", or `?`. It works like `tell`, `!`, but `!` does not return anything, while `?` returns a `Future[Msg]`. A Future by itself can be successful or failed, so we do not need to use `Either` to handle errors.
> - This way, instead of the aforementioned handling of response messages, we can do this on the sender's side:

```
val rspFuture = actorRef ? message
rspFuture.foreach { rsp => ... }
```

> - Unfortunately, we did not simply get rid of the complexity. We just took a broom and swept the complexity over to the receiver's side. And what is worse, the complexity is not only something we need to handle in our implementation of the Actor model; it also leaks into the user code.
> - Our `ask` method will look something like this:

```
def ask(msg: Msg): Future[Msg] = {
  val p = PromiseMsg
  queue.addOne((msg, p)) // Queue[Msg, Promise[Msg]]
  p.future
}
```

> Our queue of messages becomes a queue of tuples: messages plus promises. But we also need to take into account that the user will still want to use the bang method and not wait for the response, so:

```
def bang(msg: Msg): Unit = queue.addOne((msg, None))
def ask(msg: Msg): Future[Msg] = {
  val p = PromiseMsg
  queue.addOne((msg, Some(p))) // Queue[Msg, Option[Promise[Msg]]]
  p.future
}
```

- Then, in `onMessage`, the user needs to call `p.success` or `p.failure` to send the response back through a future. And this is a risky thing: it is easy for the user to introduce a bug by forgetting about it or by doing it in a wrong way.
- We can help the user a bit with helper methods. For example, we can take advantage of implicit parameters:

```
def onMessage(msg: Msg)(using p: Option[Promise[Msg]]): Behavior[Msg] = {
  case msg => ...; respond(Done)
}

private def respond(rsp: Msg)(using p: Option[Promise[Msg]]): Behavior[Msg] = {
  p.foreach(_.success(rsp))
  Behavior.same
}
```

This way, the user sees `p` in the `onMessage` declaration, but does not have to use it at all. They only need to call the helper method `respond` with the right response as the last expression in the given case. The compiler can even help us here, because without that call to `respond` the code will not compile.

23. Implementation in progress

    - I am currently working on implementing the Actor model from scratch. The project is called `signals3`, and it is an event streaming library. I want to add actors to it. My work in progress is on the `actors` branch.
    - It is a bit different in a few places from what I have talked about today. Some parts of it are not ready yet, and two big features (switching between behaviors and how the internal state is handled) are deliberately different from the standard Actor model.
    - Everything is documented in comments and in associated Markdown files. There are lots of unit tests, and the history of commits shows the progress. If this lecture was interesting to you, I invite you to take a look.